EPSR-test-ex122 Diffuse scattering (diffuse, sharm_diffuse)

Program(s): epsr | chained passes: 2 | verification: ALL IDENTICAL (all 2 passes)

What this case tests

Exercises the `diffuse` option of the `epsr` engine [1,2,3]: for a crystal, the scattering at non-Bragg reciprocal-lattice points is expanded in spherical harmonics and averaged over Q bins and over runs.

Code exercised

Fortran (original, fortran_src/):

- `ato_manipulation_rev4.f`: `sharm_diffuse`, `initialise_diffuse_values` (module `diffuse_mod`), `bragginten`
- `read_and_save_files.f`: `outgofzs_sharm` (running averages of the coefficients)
- `epsr_standalone_rev3.f`: `runepsr17` (`diffuse` keyword)

Rust (port, rust/epsr-core/src/):

- `bragg.rs`: `Bragg::read_keywords`, `bragginten`, `sharm_diffuse`
- `tools/sharm_routines.rs`: `outgofzs_sharm`

Physical system

The DMSO/water crystal of ex120: 5 × 4 × 4 cubic 6.84 Å unit cells of 2 DMSO + 4 water in an orthorhombic 34.20 × 27.36 × 27.36 Å box (480 molecules, 2560 atoms, 21 atom-type pairs, 300 K). The atom types are those of EPSR-test, so its `.pcof`, weights and three D-DMSO/water neutron data files are used unchanged; for the crystal they are numerical input only, not a physical model of measured data. With `diffuse 2 2 1 4 2 1` the non-Bragg points (h or k odd) are expanded in spherical harmonics with l = 0, 2, 4 and m = 0..l [4]; only (h,k,0) reflections are used.

Inputs

Outputs of a Fortran `epsr` pre-run with `iinit 1` (accumulators, `.ato`, `.erg`, `DIFFUSE.h01` files), keeping only the `d01` and `d02` coefficient files; `scripts/make_case.sh` regenerates them. Keywords: `ntimes 2`, `qwidthqmax 0.01 2.2`, `mplicities 5 4 4 1 1 0`, `hklqmin 0.0 10.0 0.1`, `diffuse 2 2 1 4 2 1`, `nsumt 1`, `iinit 0`.

How the accuracy was verified

`commands.txt` (1 program call, repeated 2 times, each pass reading the files written by the previous one) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; only the date line of `.out` and the wall-clock columns of `fort.35` are masked). The Fortran output is stored in `fortran_reference.tar.gz`. A run-time FORMAT in the original source (`sharm_rev2.f`/`sdf_rev2.f`, and `sharm_diffuse` for `epsr`) lacks a comma after the Hollerith `1h(` and is rejected by current gfortran, so the reference build inserts that comma (originals kept as `*.orig`); the port behaves like the patched program. Result of the final regression run of the complete port: every file identical in all 2 passes.

The reference was made with a Fortran build in which a comma is inserted in the run-time title FORMAT of `sharm_diffuse` (`1h(` without comma); with gfortran >= 14 the unpatched `epsr` stops there with "Missing comma between descriptors" in pass 1. `d01` and `d02` are read and averaged, the missing `d03` sets `nsums = -1` and switches the running averages to `nesum` mode; pass 2 starts with `nsumt -1`. Exercised per the README: `bragginten` keeping the hkl arrays, the coefficients `dlm` (gfortran intrinsics `cosd`/`sind`/`atan2d`, inversion image for h > 0), averaging per Q bin and `smood`. The `DIFFUSE.h01` files are compared with all other outputs.

Notes

`sharm_diffuse` indexes `intenhkl(nhkl,npairs)` as `intenhkl(ip,ib)`, valid only while nhkl <= npairs; `qbraggmax` keeps nhkl = 15 <= 21. The port reads the same element and reports larger cases as unsupported.

References

[1] A. K. Soper, "Empirical potential Monte Carlo simulation of fluid structure", Chem. Phys. 202, 295-306 (1996).
[2] A. K. Soper, "Partial structure factors from disordered materials diffraction data: An approach using empirical potential structure refinement", Phys. Rev. B 72, 104204 (2005).
[3] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[4] C. G. Gray and K. E. Gubbins, Theory of Molecular Fluids, Vol. 1: Fundamentals, Clarendon Press, Oxford (1984).
[5] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.